Multi-Goal Traversal Planning in Cluttered Environments via Improved A* and MMAS-VND-CL

融合改进 A* 与 MMAS-VND-CL 的障碍环境多目标点遍历规划

作者姓名 *Member, IEEE*

摘要—现代发展中，探索无人接驳与物流配送等涉及多目标遍历路径规划技术非常重要。在相关技术中，基于栅格的地图建模能够显著简化路径规划过程，尤其是在环境复杂且中间节点较多的情况。针对栅格障碍环境下的多目标点遍历问题，提出一种融合改进 A* 与 Max-Min Ant System-Variable Neighborhood Descent-Candidate List（MMAS-VND-CL）的两层协同规划框架。该框架将基于跳点搜索（JPS）的全局寻径与开放旅行商组合排序相结合，实现几何层与排序层的统一求解。几何层采用结合安全感知路径简化与弧长参数化样条平滑的改进 A* 平滑生成点对无碰撞连续路径，并构建旅行商代价矩阵；排序层提出 MMAS-VND-CL 算法以虚拟节点法求解起点与终点固定的开放旅行商问题。MMAS-VND-CL 的一个关键特征是候选列表加速的可变邻域下降局部搜索，该机制将邻域评估限制在近邻范围，在较大目标规模下仍能高效精化回路；同时融合最大-最小蚁群系统的信息素边界以维持搜索多样性，蚁群系统式伪随机转移平衡探索与利用，并以 S 形渐进调度控制局部搜索的投入时机。为评估所提框架的有效性，在三张不同障碍形态的栅格地图上对改进 A* 进行了性能测试，并对 MMAS-VND-CL 开展了参数敏感性分析、消融实验与求解器对比实验。结果表明，改进 A* 表现更好，相较于传统 A* 算法，其在 Map 3 上的规划路径长度降低 5.3%，规划时间减少 75.9%。在针对 MMAS-VND-CL 的对比实验中，MMAS-VND-CL 在 Map 1 上较传统 ACO 得到的平均路径长度降低 10.0%，运行时间减少 57.4%。

***Index Terms*—多目标点遍历，路径规划，蚁群优化，开放旅行商问题**

I. 引言

多目标点遍历路径规划是自主移动机器人领域的一项基本问题，在无人公交接驳、物流仓储管理、基础设施巡检以及搜救行动等场景中具有广泛应用 [1]–[5]。在完成例如接送乘客、检查设备、投递物品等特定任务中，机器人需要从指定起始位置出发，依次访问一系列中间目标点，最终到达目标终点。规划器需解决两个相互耦合的子问题：一是在杂乱环境中寻找连续点对之间的无碰撞路径，该子问题属于几何问题，需要在物理空间中对障碍物几何形状和路径连续性进行推理；二是确定中间目标点的最优访问顺序，该子问题是一个组合优化问题，类似旅行商问题（TSP）这类经典 NP-hard 问题。这两个子问题相互影响，任意两点之间的通行代价取决于其间存在的障碍物，而最优访问顺序又取决于这些点对间的通行代价。

总体来看，现有多目标导航方法通常仅独立处理其中一个子问题。大量旅行商问题研究假设点对间的代价由欧氏距离或已知距离矩阵给出 [6], [7]，忽略了在障碍物丰富的环境中，两点之间的最短无碰撞路径可能远长于直线距离这一事实 [8]。另一方面，例如 A*、Dijkstra、快速扩展随机树（RRT）及其变体路径规划领域的成熟算法能够高效地计算栅格地图中点对之间的最优或近似最优无碰撞路径，但这些规划器并不优化多目标点的访问顺序 [9]–[11]。少量研究在特定几何假设下探索了带障碍物的旅行商问题变体（如带障碍物的 TSP、Dubins TSP），但目前尚缺乏一种将基于栅格的全局规划器与高性能旅行商求解器相集成的、通用的障碍环境多目标点遍历框架。

针对上述约束并最大化利用两类算法的优势，本文旨在提出一种面向障碍环境多目标点遍历路径规划的协同优化框架。该框架由两个算法层组成，通过预计算

本文作者单位与通讯信息待补充。
稿件收稿日期：2026 年；修订日期：2026 年。

的代价矩阵连接。第一层是基于跳点搜索（JPS）的改进 A* 全局规划器，其利用栅格对称性跳过冗余节点扩展、仅扩展跳点，从而大幅缩减搜索树规模；该规划器结合了安全感知的路径简化流程以及弧长参数化三次样条平滑管线。第二层是面向开放旅行商问题的改进蚁群优化（ACO）求解器，其通过虚拟节点（哑节点）法将开放旅行商问题转化为标准闭环旅行商问题，从而充分利用闭环旅行商问题的搜索算子；该求解器融合了最大-最小蚁群系统（MMAS）的信息素管理、蚁群系统（ACS）风格的伪随机转移规则、以 k 近邻候选列表加速的可变邻域下降（VND）局部搜索引擎，以及基于 S 形函数的渐进式局部搜索调度和自适应停止准则。

针对上述问题，本文构建由点对路径搜索和目标访问顺序优化组成的两层规划框架。前者根据障碍分布生成可行路径并计算相应代价，后者以该代价矩阵为输入确定目标点的访问顺序。在此基础上，分别对路径处理流程和蚁群排序求解器进行改进，主要贡献如下。

1) 提出一种面向固定起终点多目标遍历的分层求解流程。将障碍约束下点对路径的连续轨迹长度作为目标点之间的通行代价，构建连接几何路径规划与访问顺序优化的代价矩阵，以减少采用欧氏距离排序时对障碍绕行成本的忽略。
2) 构建基于 JPS 的改进 A* 路径规划及后处理流程。结合平局处理、搜索失败回退、安全距离感知路径简化与弧长参数化三次样条平滑，在减少节点扩展的同时生成用于代价计算的连续路径，并通过栅格地图实验分析其规划效率与路径长度。
3) 设计 MMAS-VND-CL 访问顺序求解器。采用虚拟节点处理开放 TSP 的固定端点约束，融合 MMAS 信息素边界、ACS 式伪随机转移、候选列表加速 VND、S 形调度及自适应停止准则，并通过参数敏感性和消融实验分析各组件对解质量、稳定性与运行时间的影响。

本文其余部分组织如下：第 II 节综述全局路径规划、旅行商元启发式方法及带障碍物多目标点遍历的相关研究；第 III 节形式化问题定义；第 IV 节详述方法论，包括改进的 A* 规划器、路径后处理管线、改进的蚁群优化求解器和协同优化框架；第 V 节报告四个评估维度的实验结果：全局路径规划对比、参数敏感性分析、消融实验以及不同旅行商求解器的多目标点遍历对比；第 VI 节总结全文并展望未来工作。

II. 相关工作

A. 现有路径搜索方法及其局限

全局路径规划旨在已知静态环境中寻找两点之间的无碰撞路径，经典图搜索算法是栅格规划的主流方法 [3], [12], [13]。Dijkstra 算法以均匀方式扩展，在大地图上效率较低 [10]；A* 以启发式引导搜索并在 $h(n)$ 可采纳时保持最优性 [9]，但固定的启发式权重存在权衡，权重为 1 时扩展节点过多，加大权重则牺牲最优性 [14]。加权 A* 与任意时间修复 A* 等变体在整个搜索过程中施加统一权重 [14], [15]，而实用的权重直观上取决于到目标的距离。跳点搜索（JPS）利用栅格对称性仅扩展跳点，大幅压缩搜索树 [16]–[18]，近期变体进一步引入双向搜索与任意角度精化 [19]–[21]，本文的改进 A* 即建立在这一原理之上。采样类规划器如 RRT [11]与 PRM 提供概率完备性，在高维构型空间中表现良好，但在栅格环境中的解路径通常更长且一致性较差。

栅格路径的后处理是成熟的技术实践，包括共线性或视线裁剪 [22]以及三次样条、B 样条、回旋曲线等平滑方法 [23]–[25]。然而，裁剪和平滑通常被视为独立的事后步骤，未被纳入多目标代价的计算；将后处理得到的连续路径长度用于指导组合优化，是本文框架的一个显著特征。

B. 现有访问顺序优化方法及其不足

旅行商问题是组合优化领域中研究最为深入的问题之一 [6], [7]，精确算法通常仅适用于小规模实例，更大规模通常采用元启发式方法，如遗传算法 [26]与蚁群优化 [27], [28]。蚁群系统（ACS）的伪随机比例转移规则 [29]与最大-最小蚁群系统（MMAS）的显式信息素上下界 [30]分别改善了收敛速度与搜索稳定性，本文的排序求解器即以二者为组件基础。

对本文问题而言，现有排序方法存在两方面不足。其一，主流求解器默认点对代价为欧氏距离或已知距离矩阵，而在障碍环境中真实通行代价由避障路径决定。其二，将 2-opt、可变邻域下降（VND）等局部搜索嵌入蚁群虽可显著提升解质量 [31]–[33]，但 VND 对每个算子评估 $O(K^2)$ 个候选移动，在 $K > 100$ 时成为瓶颈，候选列表技术可将近评估限制在最近邻范围内加以缓解 [34]。此外，局部搜索的渐进调度受到关注较少，多数实现全程对固定比例的蚂蚁施加局部搜索；近期蚁群变体多关注并行化、小生境与可持续性机制 [35]–[38]。

针对上述不足，本文以安全弧长计价纳入障碍代价，以候选列表加速 VND（第 IV-C4 节）将局部搜索复杂度降至 $O(K \cdot k)$，并以 S 形调度（第 IV-C5 节）安排局部搜索的投入时机。

C. 障碍约束下的两类方法结合

路径规划与旅行商优化的交叉领域已在多个名义下得到探索 [6], [7]。在连续平面多边形障碍下，点对通行代价依赖避障几何而非欧氏度量 [8]；Dubins 变体考虑最小转弯半径约束，邻域或贴近变体则将目标从点泛化为区域 [39]。这些公式依赖连续几何表示并假设特定的障碍形状，难以直接用于障碍以任意占用模式表示的栅格导航。

结合路径规划器与旅行商求解器的两层架构已有若干先例 [1], [2]。然而，这些方法通常采用现成的规划器和求解器，未针对多目标点遍历上下文优化任一层。代价矩阵由原始栅格路径计算得出，因离散化伪影而高估了真实路径长度，且旅行商求解器缺乏我们框架所提供的例如开放旅行商编码、候选列表加速等此类问题特定增强。

开放旅行商变体——其中起点和终点为不重合的固定位置——与自主导航任务尤为相关，机器人从仓库出发并到达目的地。经典旅行商问题文献的大部分内容，包括大多数 TSPLIB 基准实例，针对的是闭环旅行商问题。开放旅行商问题需要不同的编码和代价评估函数，我们在第 III-B 节中将其形式化，并在第 IV-C1 节中通过虚拟节点法给出算法实现，该方法类似于开放车辆路径问题中所采用的技术 [40]。

综上，A* 类方法可给出高质量的点对点路径，开放式旅行商元启发式则擅长组合排序，二者单独使用均难以覆盖障碍环境下的完整多目标点遍历任务。本文的定位是将跳点搜索全局规划、安全感知后处理与开放旅行商蚁群优化集成为一条完整的协同管线，并在组件层面做相应工程化改进。相较于既有集成先例 [1], [2]，本文以安全感知后处理的连续轨迹弧长而非原始栅格路径长度构建代价矩阵，并在统一几何接口下对排序模块的多种实现进行了对照，各增量的实验归因见第 V 节。

III. 问题形式化

本节将障碍环境多目标点遍历路径规划问题进行形式化定义。首先建立基于栅格的环境模型，然后定义具有固定起点和终点的开放旅行商问题，最后给出完整的优化目标。

A. 栅格地图与障碍物表示

运行环境建模为 $n \times n$ 单元的二维方形栅格地图，$r \in \{1, \dots, n\}$ 与 $c \in \{1, \dots, n\}$ 分别为行索引和列索引。每个栅格单元 $\mathbf{M}(r, c)$ 取 0 或 1，分别代表自由空间和被占用空间。假设地图在规划前静态且完全已知。

对于对角移动，相邻的两个正交栅格也必须同时为自由空间；否则机器人将穿越障碍物角点，这对非点状机器人来说在物理上不可行。

栅格坐标 (r, c) 按 $x = c - 0.5$、$y = r - 0.5$ 映射为连续世界坐标 (x, y)，连续坐标系原点置于栅格左下角（栅格 $(1, 1)$ 映射为 $(0.5, 0.5)$），与占用栅格地图中机器人定位的标准约定一致。

B. 具有固定起点和终点的开放旅行商问题

设任务规格包含一个起点 S、K 个中间目标点 $\mathcal{T} = \{T_1, T_2, \dots, T_K\}$ 和一个终点 G。所有点以栅格坐标 $[r, c]$ 指定，完整点集记为 $\mathcal{P} = \{p_1, \dots, p_N\}$（$N = K + 2$），其中 $p_1 = S$，$p_i = T_{i-1}$（$i = 2, \dots, K + 1$），$p_N = G$。起点和终点不重合，占据任何可行解的边界位置。

一个有效路径由访问顺序 $\boldsymbol{\pi} = (\pi_1, \pi_2, \dots, \pi_N)$ 定义，该顺序是索引集 $\{1, 2, \dots, N\}$ 的排列，满足边界约束：

$$\begin{cases} \pi = 1, & \text{Start node} \\ \pi = N, & \text{End node} \end{cases} \tag{1}$$

中间索引 $\{\pi_2, \dots, \pi_{N-1}\}$ 构成 $\{2, \dots, N-1\}$ 的一个排列，表示目标点的有序访问。这是**开放旅行商问题**的形式化，区别于经典闭环旅行商问题——后者的路径构成环路且起始节点可任意选取 [6], [7]。

C. 优化目标

对于任意有序点对 (p_i, p_j)，设 $\mathcal{P}(p_i, p_j)$ 表示从 p_i 到 p_j 在栅格地图 $\mathbf{M}$ 中的最短无碰撞路径（由全局路径规划器计算的栅格单元序列），其长度 $L\big(\mathcal{P}(p_i, p_j)\big)$ 为序列中各边代价之和。若 p_i 和 p_j 之间不存在无碰撞路径，则 $\mathcal{P}(p_i, p_j) = \varnothing$，$L(\varnothing) = \infty$。

路径 $\boldsymbol{\pi}$ 的总代价为 $N - 1$ 段连续路径长度之和：

$$C(\boldsymbol{\pi}) = \sum_{k=1}^{N-1} L\big(\mathcal{P}(p_{\pi_k}, p_{\pi_{k+1}})\big) \tag{2}$$

障碍环境多目标点遍历问题表述为：

$$\begin{aligned}\boldsymbol{\pi}^* &= \arg\min_{\boldsymbol{\pi}\in\Pi} C(\boldsymbol{\pi}) \\ \text{s.t.}\quad & \pi_1 = 1,\ \pi_N = N \\ & \{\pi_2,\ldots,\pi_{N-1}\} = \{2,\ldots,N-1\} \\ & \mathcal{P}(p_{\pi_k}, p_{\pi_{k+1}}) \neq \varnothing \quad \forall k \in \{1,\ldots,N-1\}\end{aligned} \tag{3}$$

其中 Π 为所有满足边界约束的 $\{1,\ldots,N\}$ 排列的集合。式 (3)中的优化耦合了由全局规划器求解为每段路径寻找最短无碰撞路径和由旅行商求解器求解确定中间目标点的最优排序这两个互相作用的子问题。代价矩阵元素 $\mathbf{D}(i,j) = L(\mathcal{P}(p_i,p_j))$ 构成两个子问题之间的接口，将完整的障碍约束距离拓扑编码为 $N\times N$ 矩阵。该解耦是精确的：对于给定的代价矩阵 $\mathbf{D}$，式 (3)的优化退化为开放旅行商问题；对于给定的访问顺序 $\boldsymbol{\pi}$，每段路径是独立的全局规划问题。本文实现中，$L\big(\mathcal{P}(p_i,p_j)\big)$ 以简化与平滑后连续轨迹的欧氏弧长计算，使代价矩阵更贴近实际执行路径；上述解耦精确性对任意一致的点对距离度量均成立。

在特殊情况无中间目标点下，问题退化为单对最短路径查询：$\boldsymbol{\pi}^* = (1,N)$，$C(\boldsymbol{\pi}^*) = L(\mathcal{P}(S,G))$，即经典单对路径规划问题。

IV. 方法论

为克服单一路径规划器无法优化访问顺序、以及无约束旅行商模型忽略障碍代价的局限，本节给出由路径规划与排序两阶段组成的协同框架：改进 A* 全局规划器搜索点对无碰撞路径，并经安全简化与样条平滑形成障碍约束代价矩阵；MMAS-VND-CL 求解器在该矩阵上确定最优访问序列。两阶段经代价矩阵耦合，完成从几何寻径到组合排序的完整流程。算法超参数除特别说明外统一列于表 I。第 IV-A–IV-B 节介绍全局规划及后处理，第 IV-C 节详述开放旅行商求解，第 IV-D 节给出两层协同设计。

A. 改进的 A* 全局路径规划器

1) 跳点搜索： 传统 A* 需逐格扩展邻居，在大规模开阔区域中节点扩展量与地图面积成正比。为此本文采用基于跳跃点搜索（Jump Point Search, JPS）的改进 A* 算法 [16], [17]。JPS 利用栅格对称性做邻居剪枝，按扩展方向仅保留自然邻居与因障碍产生的被迫邻居，跳跃函数 $\mathrm{jump}(r,c,dr,dc)$ 沿方向推进至到达目标、遇障碍或发现被迫邻居为止，中间节点不进入开放集；跳点成本按曼哈顿或对角距离累计；搜索完成后经父指针回溯并在相邻跳点间直线插值补全路径。剪枝规则示意如图 1。本文在此基础上做三方面改进。

由于 JPS 的跳跃特性，重建路径中的对角线移动可能穿越障碍物拐角。为此，本文算法在路径重建阶段检测此类情形，并将对应移动替换为沿栅格边界的 L 形路径，从而避免碰撞。

2) 打破平局策略： 当开放集中的多个节点具有相同的 f 值时，标准 A* 的选择顺序取决于具体实现（通常是先入先出或随机），本质上是任意的。然而，在 f 值相同的节点中，h 值较小的节点意味着其实际代价 $g = f - h$ 较大，即该节点距起点更远、更接近目标。优先扩展此类节点可将搜索方向导向目标，且不影响最优性保证。为此，本文提出的改进 A* 算法采用 “f 值相同时优先选择 h 值更小的节点” 的打破平局策略，节点比较规则定义为：

$$\begin{cases} n_1, & f(n_1) < f(n_2) \\ n_2, & f(n_1) > f(n_2) \\ n_1, & f(n_1) = f(n_2) \wedge h(n_1) < h(n_2) \\ n_2, & f(n_1) = f(n_2) \wedge h(n_1) > h(n_2) \end{cases} \tag{4}$$

从物理意义上理解，由评估函数 $f(n) = g(n)+h(n)$ 可知，当两个节点的 f 值相同时，g 值较大者表示该节点距起点更远，而 h 值较小者表示其更接近目标。优先扩展此类节点使搜索偏向目标方向，减少了远离目标区域的无效探索。

B. 路径后处理

JPS 搜索得到的路径由栅格中心点序列组成，存在路径包含冗余的中间点以及由离散的水平和垂直段组成导致平滑性不足的问题。本节提出两阶段处理方式来解决这些问题。

1) 安全感知路径简化： 传统例如拐角裁剪的路径简化方法直接连接路径上的非相邻节点，跳过中间点以减少路径点数量。然而，这类方法通常仅检查连接线段是否穿越障碍物，忽略了路径点与障碍物之间的安全裕度，在障碍物密集区域可能导致路径过于贴近障碍物。为此，本节提出一种安全距离感知的路径简化算法，其核心思想是在简化过程中保持路径与障碍物之间的最小安全距离。该算法采用四步流程处理输入路径：首先

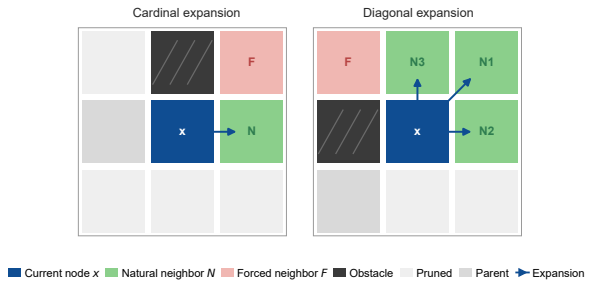


图 1. JPS 邻居剪枝规则示意图。

从路径中识别方向发生变化的节点。在保留起点与终点的前提下，计算中间节点与前后节点的连接方向，并标记前向连接方向与后向连接方向不同的节点为拐点 C_i。其次从当前拐点 C_i 出发，正向扫描后续拐点 C_j（$j > i$），通过安全性检查逐一验证线段 $C_i \to C_j$ 是否满足安全距离约束，跳过中间的拐点并记录最远的可达拐点 j^* 为新的锚点。这种贪心策略在保证安全性的前提下最大化了单步简化程度。在此基础上，贪心步骤确定最远可达拐点 j^* 后会检查被跳过的拐点 k（$i < k < j^*$）能否通过安全线段直达更远处的拐点 j'（$j' > j^*$），如果可以则记录路径 (k, j')。随后比较两条路径的长度，假设路径 A 为 $d(C_i, C_{j^*}) + d(C_{j^*}, C_{j'})$，即先到达 j^* 再继续到 j'；路径 B 为 $d(C_i, C_k) + d(C_k, C_{j'})$，即跳过 j^* 改经 k 路由。若路径 B 更短，则保留中间拐点 C_k；否则仅保留贪心结果 C_{j^*}。这一比较确保中间点探索不会以牺牲路径效率为代价。算法 1给出了路径简化算法的伪代码。

其中，安全性检查函数的实现基于逐栅格采样：从两个节点 p_1 到 p_2 沿线段密集采样，对每个采样点计算其与最近障碍物栅格边界的距离。只要任意采样点的距离小于安全阈值 d_{safe}，即判定该连接不安全。点到栅格边界的距离计算公式为：

$$\begin{aligned} d_x &= \max\left(0,\ |x_p - x_c| - 0.5\right) \\ d_y &= \max\left(0,\ |y_p - y_c| - 0.5\right) \end{aligned} \tag{5}$$

$$d(p, \text{cell}) = \sqrt{d_x^2 + d_y^2} \tag{6}$$

公式中，$p = (x_p, y_p)$ 为上述栅格—连续坐标映射下的采样点，(x_c, y_c) 为障碍物栅格的中心坐标。$\max(0, \cdot)$ 算子处理采样点在某坐标轴上已落入栅格水平或垂直范围内的情况。当计算距离 $d(p, \text{cell})$ 小于设定安全距离阈值 d_{safe} 时，表示采样点已进入障碍物栅格的安全禁区。

Algorithm 1 Safety-distance-aware path simplification

Input: grid path $P = [P_1, \ldots, P_N]$, occupancy grid, safety margin d_{safe}
Output: simplified path P_s ($|P_s| \le N$)

$C \leftarrow$ the corner points of P where the direction changes, plus the two endpoints;
$P_s \leftarrow [C_1]$; $i \leftarrow 1$;
while $i < |C|$ **do**
  $j^* \leftarrow$ the farthest corner reachable from C_i by a collision-free segment; *greedy forward scan*
  if $j^* = |C|$ **then**
    append $C_{|C|}$ to P_s and break;
  end if
  $(k^*, j') \leftarrow$ among all skipped corners k ($i < k < j^*$) and farther corners j ($j^* < j \le |C|$), the pair with the largest j such that the segment $C_k \to C_j$ is collision-free;
  if k^* exists and $d(C_i, C_{k^*}) + d(C_{k^*}, C_{j'}) < d(C_i, C_{j^*}) + d(C_{j^*}, C_{j'})$ **then**
    append C_{k^*} and $C_{j'}$ to P_s, and set $i \leftarrow j'$; *route through the intermediate corner*
  else
    append C_{j^*} to P_s, and set $i \leftarrow j^*$;
  end if
end while
return P_s;

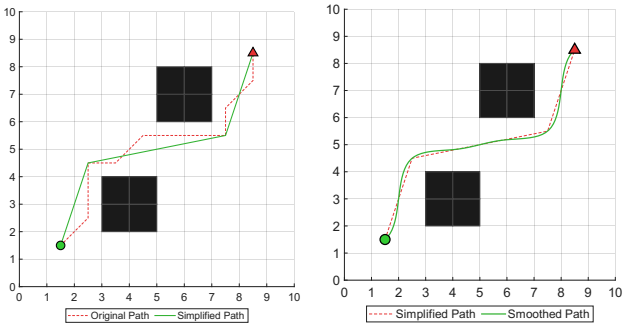
2) 弧长参数化三次样条平滑: 简化路径虽已去除冗余点，但输出路径不够平滑，难以反应实际运动的路程。为此，本文采用弧长参数化三次样条平滑，以路径点间的累积欧氏距离而非点索引作为参数，使参数空间与实际几何空间保持一致。平滑过程第一步是稀疏段致密化。对于路径中相邻点间距超过 2 个栅格单位的稀疏段，在两端点之间沿线段线性插值 $\lfloor \ell/2 \rfloor$ 个中间点。致密化的目的是为样条曲线提供足够的控制点，防止曲线在长直段处因控制点不足而产生 Runge 现象造成曲线偏离预期折线甚至裁剪障碍物角点。第二步进行弧长参数化。设致密化后的路径点序列为 $\{P_0, P_1, \ldots, P_M\}$，其中 $P_i = (x_i, y_i)$ 为连续坐标。计算各点的累积弦长参数，即 $s_0 = 0$，$s_i = s_{i-1} + \|P_i - P_{i-1}\|_2$（$i = 1, 2, \ldots, M$）。然后分别以 s 为自变量对 $x(s)$ 和 $y(s)$ 进行三次样条插值，得到参数化曲线 $(x(s), y(s))$。三次样条在每个子区间 $[s_i, s_{i+1}]$ 上为三次多项式，保证一阶和二阶导数连续，从而确保曲线的曲率连续性。第三步进行等弧长重采样。以插值密度 ρ 沿弧长参数 s 均匀采样，生成最终的平滑路径点序列。由于 s 与实际几何距离近似线性关系，等弧长采样保证了路径点在空间中的均匀分布，避免了传统等参数采样在曲率大处点密、曲率小处点疏的问题。算法 2给出了平滑算法的伪代码。

Algorithm 2 Arc-length parameterized cubic spline smoothing

Input: grid path $P = [P_1, \ldots, P_N]$, density ρ (default 10)
Output: smoothed continuous path $Q = [Q_1, \ldots, Q_M]$ in $[x, y]$

convert each grid point $P_i = (r_i, c_i)$ to continuous coordinates $(x_i, y_i) = (c_i - 0.5,\ r_i - 0.5)$;
for each consecutive pair whose segment length exceeds 2 **do**
  insert $\lfloor \text{length}/2 \rfloor$ equally spaced intermediate points along the segment; *densification to prevent spline overshoot*
end for
compute the cumulative chord length $t_1 = 0$, $t_i = t_{i-1} + \|P_i - P_{i-1}\|_2$; *arc-length parameter*
remove duplicate t values to keep strict monotonicity;
fit cubic splines $x(t)$ and $y(t)$ through the densified points;
sample $x(t)$ and $y(t)$ at ρ points per segment to obtain Q;
return Q;

3) 流水线集成: 上述后处理操作以流水线方式集成。JPS 搜索输出原始栅格路径 → SimplifyPath 去除冗余点 → SmoothPath 生成平滑的连续坐标路径。整个流水线一次性完成，输出路径兼具安全裕度与平滑性。这种两阶段设计的合理性在于路径简化和样条平滑分别解决了路径质量的两个不同维度。安全感知路径简化减少了路径点数量并降低了后续计算的复杂度，同时保证了路径的安全裕度；平滑则将离散的栅格路径转化为连续曲线，使机器人能够以平滑的运动轨迹跟踪路径。若跳过简化直接进行平滑，样条曲线需要处理大量冗余的控制点，不仅计算成本更高，还可能因控制点过于密集而产生不稳定的曲线形状。路径后处理前后对比如图 2所示。



(a) 原始路径与简化后路径　(b) 简化后路径与平滑后路径

图 2. 路径通过管线各阶段的对比。

C. 改进的开放旅行商蚁群优化

本框架的排序层在全局规划层产生的代价矩阵上求解开放旅行商问题。给定由固定起点 S、K 个中间目标点 $\{T_1, \ldots, T_K\}$ 和固定终点 G 组成的 N 个点，目标是找到使总路径代价最小的 K 个目标点排列。我们在蚁群优化（ACO）元启发式方法基础上引入六个集成机制，本小节将会逐一展开描述。

1) 虚拟节点法编码开放旅行商问题: 对于开放旅行商问题，起点 S 与终点 G 为任务给定的不同物理位置，优化仅针对 K 个中间目标点进行排列。朴素的全排列编码会产生首尾错位的无效解；若在编码中固定首尾，则 S、G 被排除在信息素模型与局部搜索之外，限制对完整解的优化能力。为此，本文采用虚拟节点法将开放旅行商问题转化为标准闭环旅行商问题 [40]。引入虚拟节点 $V = N + 1$，构造 $(N+1) \times (N+1)$ 的扩展代价矩阵 $\mathbf{D}_{\text{ext}}$：

$$\mathbf{D}_{\text{ext}}(i,j) = \begin{cases} 0, & \{i,j\} = \{V, 1\} \\ 0, & \{i,j\} = \{V, N\} \\ \text{BIG}, & (i = V \vee j = V) \wedge \\ & \{i,j\} \cap \{1, N\} = \varnothing \\ \mathbf{D}(i,j), & \text{otherwise} \end{cases} \tag{7}$$

其中 $\text{BIG} = 2(N+1) \cdot \max_{i,j} \mathbf{D}(i,j)$ 为严格大于任何可行回路代价的惩罚值，确保虚拟节点在任何近优回路中不直接连接任何中间目标点，从而强制虚拟节点仅与起点和终点相邻。

求解器在 $\mathbf{D}_{\text{ext}}$ 上按标准闭环旅行商问题运行，信息素矩阵 $\boldsymbol{\tau}$ 覆盖完整的 $(N+1) \times (N+1)$ 空间。求解器收敛后从最优回路中去掉 V，按回路顺序读取剩余节点并调整为起点在前、终点在后，即得开放访问顺序。相对边界固定编码，该转换使起终点完整参与信息素更新与局部搜索，闭环旅行商算子可统一使用，代价评估即为 $\mathbf{D}_{\text{ext}}$ 上的回路求和。启发式可见度矩阵 $\boldsymbol{\eta}$ 定义为

$$\eta_{ij} = \begin{cases} \text{BIG}, & \{i,j\} = \{V,1\} \text{或} \{V,N\} \\ \dfrac{1}{\mathbf{D}_{\text{ext}}(i,j)}, & \mathbf{D}_{\text{ext}}(i,j) > 0 \\ 0, & \text{otherwise} \end{cases} \tag{8}$$

即对涉及 V 的零代价边赋予大启发式值，强烈吸引 V 连接其所需邻居；对其他点对取扩展代价的倒数。闭合

回路 $\boldsymbol{\sigma} = (\sigma_1, \ldots, \sigma_{N+1})$ 的代价为含回路边 $\sigma_{N+1} \to \sigma_1$ 的各边之和。

2) 具有动态边界的最大-最小蚁群系统信息素更新： 最大-最小蚁群系统（MMAS）[30]将信息素限制在显式区间 $[\tau_{\min}, \tau_{\max}]$ 内以防止单边支配与灭绝，并聚焦最优解沉积。本文采用该机制，并给出与代价尺度匹配的边界初始化与更新顺序。

每次迭代先对所有信息素蒸发 $\tau_{ij} \leftarrow (1-\rho)\cdot\tau_{ij}$（$\rho$ 为蒸发率），再在当代最优回路 $\boldsymbol{\sigma}_{\text{iter-best}}$ 与全局最优回路 $\boldsymbol{\sigma}_{\text{global-best}}$ 上分别沉积

$$\tau_{ij} \leftarrow \tau_{ij} + \frac{Q}{C_{\text{iter}}}, \qquad \tau_{ij} \leftarrow \tau_{ij} + \frac{Q}{C_{\text{global}}} \tag{9}$$

其中 Q 为沉积常数，C 为对应回路代价。沉积量与回路代价成反比，且对称作用于 (i,j) 与 (j,i)。沉积后将信息素限幅到当前区间，边界由全局最优代价动态给出：

$$\tau_{\max} = \frac{1}{\rho \cdot C_{\text{global}}}, \qquad \tau_{\min} = \frac{\tau_{\max}}{N+1} \tag{10}$$

初始信息素取均匀值 τ_0，对应边界 $\tau^0_{\max} = 1/(\rho\cdot\tau_0\cdot(N+1))$ 与 $\tau^0_{\min} = \tau^0_{\max}/(5(N+1))$，使 τ_0 在起步阶段不被限幅截断；另设硬下限 $\tau_{\min}$，防止小尺度代价下低频边下溢灭绝。单次迭代的更新顺序为：蒸发、按式 (9)沉积、限幅、按式 (10)重算边界。边界的动态更新至关重要，随着搜索发现更优回路，C_{global} 减小使 $\tau_{\max}$ 增大，允许算法加强对当前最优回路的置信；比值 $\tau_{\min}/\tau_{\max} = 1/(N+1)$ 保持恒定相对间隔，为所有边保留基线探索概率。

3) 蚁群系统风格的伪随机转移规则： 蚁群系统（ACS）的伪随机比例规则 [29]以可控概率贪心选择最优候选、其余情形按轮盘探索，平衡收敛与多样性。本文采用该规则作为路径构建阶段的转移机制。具体实现时，当蚂蚁在当前节点 u 从未访问候选集 $\mathcal{U}$ 中选择下一节点时，首先计算每个候选的得分 $\text{score}(v) = (\tau_{uv})^{\alpha}\cdot(\eta_{uv})^{\beta}$（$\alpha$ 和 β 分别为信息素和启发式权重指数）。随后以概率 q_0 进行利用，确定性地选择得分最高的候选 $v_{\text{next}} = \arg\max_{v\in\mathcal{U}} \text{score}(v)$；以概率 $1-q_0$ 进行探索，执行轮盘选择，每个候选 v 以 $\text{score}(v)/\sum_{u\in\mathcal{U}}\text{score}(u)$ 的概率被选中。若所有得分为零即所有剩余节点不可达，则随机选择一个未访问候选作为回退。可以看出，q_0 越大，越多的构建步采用贪心选择以加速收敛；q_0 越小，则随机探索的比重越高。障碍约束代价矩阵中的点对距离通常具有空间局部性，因而启发式 $\boldsymbol{\eta}$ 能够提供较可靠的引导，这使得贪心概率在保证多样性的前提下仍有利于收敛。

4) 具有候选列表加速的可变邻域下降局部搜索： 蚂蚁所构建的路径虽受信息素与启发式信息引导，但仍未必达到局部最优。一个自然的改进思路是：在每一代对部分蚂蚁构造的回路施加局部搜索，通过系统考察邻域排列来进一步精化解的质量。可变邻域下降（Variable Neighborhood Descent, VND）[31]正是这样一类方法，它依次调用多种邻域结构，每种结构对应一类路径变换，并在当前结构无法继续改进时切换到下一种，直至所有邻域均无法产生更优解。已有研究表明，将 MMAS 或 ACS 与 VND 混合在车辆路径问题中效果显著 [32]。不过，标准 VND 在每个算子上需考察 $O(K^2)$ 个邻域，当目标点规模增大时计算开销会迅速上升，成为主要瓶颈。进一步观察发现，这类考察中大量尝试并不奏效：相距较远的节点对参与 2-opt 交换时，往往很难带来回路代价的改善。

为此，本文先引入预计算的候选列表以过滤缺乏前景的邻域评估，该思路在旅行商问题的局部搜索中已有广泛应用 [34]。具体而言，对每个节点根据扩展代价矩阵 $\mathbf{D}_{\text{ext}}$ 预先筛选其 k 个最近邻，并以布尔矩阵加以存储。局部搜索中仅当移动所引入的新边与候选近邻相关时才计算代价变化，否则直接跳过评估。

在此基础上，本文在包含虚拟节点在内的 $N+1$ 个节点所构成的闭合回路上实施 VND，并按固定顺序交替使用三种邻域算子：

1) **2-opt**：将路径中一段连续子序列整体反转，即用边 (u,v) 与 (u^+,v^+) 替换原边 (u,u^+) 与 (v,v^+)，从而消除欧氏型代价下常见的路径交叉。
2) **重定位**：从当前位置取出某一节点，将其插入回路的其他位置，并相应平移中间节点，用于纠正节点被错误放置到不同路径区段的情形。
3) **交换**：在保持其余节点不变的前提下对调两个节点的位置，适用于两节点恰被安排到对方更优位置的情况。

上述算子在 VND 循环中均采用“首次改进”（first-improvement）策略。每轮按 2-opt、重定位、交换的固定顺序各执行一次；只要任一算子产生改进，便重新开始下一轮，当三种算子在一轮中均无法改进时，局部搜索终止。由于问题定义在闭合回路上，当交换操作跨越回路的首尾相邻节点时，以前驱、后继的循环下标处理首尾衔接，并对跨回路边的相邻交换单独计算三边代

价，而非沿用线性路径下的邻接判断。

从复杂度来看，若不采用剪枝，每个 VND 算子需考察 $O((N+1)^2)$ 个候选移动；引入候选列表后，每个节点仅需参与约 $O(k)$ 次邻域检查，单算子复杂度降为 $O((N+1)\cdot k)$。当 k 远小于 $N+1$ 时，相对全邻域考察可有效降低计算量。

算法 3给出了候选列表加速 VND 的完整伪代码，VND 邻域搜索的完整流程如图 3所示。

5) S 形渐进局部搜索调度： 即使引入了候选列表加速，VND 局部搜索仍需要较长的计算时间。若在每一代对所有蚂蚁都做局部搜索，早期迭代的资源往往会被浪费，此时种群尚未辨明搜索空间中较有希望的区域，精化低质量路径的收益有限；后期，当信息素已集中于高质量区域时，密集局部搜索对于进一步打磨解却十分关键。若始终对固定比例的蚂蚁实施局部搜索并不理想——比例偏低会减慢收敛，偏高则在尚未成熟的解上耗费算力。为此，本文在迭代过程中按 S 形（sigmoid）函数动态调整接受 VND 局部搜索的蚂蚁比例：

$$\mathrm{optRatio}(t) = y_{\min} + (y_{\max} - y_{\min}) \cdot \frac{t^a}{t^a + (1-t)^a} \quad (11)$$

式中，$t = (\mathrm{iter} - x_L)/(x_R - x_L)$ 为当前迭代在转换区间 $[x_L, x_R]$ 内的归一化位置；$y_{\min}$ 与 $y_{\max}$ 分别表示搜索初期与后期接受局部搜索的蚂蚁比例上下限，a 用于控制 S 形曲线的陡峭程度。当 t 位于 x_L 之前时，optRatio 取 $y_{\min}$；超过 x_R 之后则保持为 $y_{\max}$；在区间内部，比例沿 S 形曲线平滑过渡。

这一调度使搜索从早期广泛探索平滑过渡到后期对优质解的密集精化，同时由部分蚂蚁维持构建多样性。S 形 optRatio(t) 曲线如图 4所示。

此外，本文局部搜索蚂蚁比例采用精英与随机相结合的方式，在确定本轮接受局部搜索的蚂蚁数目 $n_{\mathrm{opt}} = \lceil n_{\mathrm{ants}} \cdot \mathrm{optRatio} \rceil$ 之后，从中取 $n_{\mathrm{elite}} = \lceil n_{\mathrm{opt}} \cdot r_{\mathrm{elite}} \rceil$ 只排名靠前的精英个体优先接受优化，以强化对优质解的精细雕琢；其余 $n_{\mathrm{random}} = n_{\mathrm{opt}} - n_{\mathrm{elite}}$ 只从非精英个体中随机抽取并同样接受优化，从而有机会发现精英之外的其他有潜力区域，在利用与探索之间保持平衡。其中 r_{elite} 为精英占比参数。

6) 自适应停止准则： 算法以最大迭代次数 n_{iter} 为预算上限，以免在困难实例上难以收敛；但较易实例往往远在此之前收敛，继续迭代只会增加计算开销，而过早停止又可能错过更优解。因此停止准则需区分“已充分收敛”与“暂时尚未改进”，同时监测两个互补指标。

Algorithm 3 Candidate-list-accelerated variable neighborhood descent

Input: closed tour $\boldsymbol{\sigma} = [\sigma_1, \ldots, \sigma_{N+1}]$, extended cost matrix $\mathbf{D}_{\mathrm{ext}}$, candidate boolean matrix $\mathbf{C}$
Output: improved tour $\boldsymbol{\sigma}$ and its cost

cost ← the cost of tour $\boldsymbol{\sigma}$ under $\mathbf{D}_{\mathrm{ext}}$;
repeat
  improved ← false;
  $[\boldsymbol{\sigma}, \mathrm{cost}, \mathrm{ok}]$ ← apply the 2-opt operator with candidate-list pruning (first-improvement);
  improved ← improved or ok;
  $[\boldsymbol{\sigma}, \mathrm{cost}, \mathrm{ok}]$ ← apply the relocate operator with candidate-list pruning;
  improved ← improved or ok;
  $[\boldsymbol{\sigma}, \mathrm{cost}, \mathrm{ok}]$ ← apply the swap operator with candidate-list pruning;
  improved ← improved or ok;
until improved = false
cost ← the exact cost of the final tour $\boldsymbol{\sigma}$;
return $\boldsymbol{\sigma}$, cost;

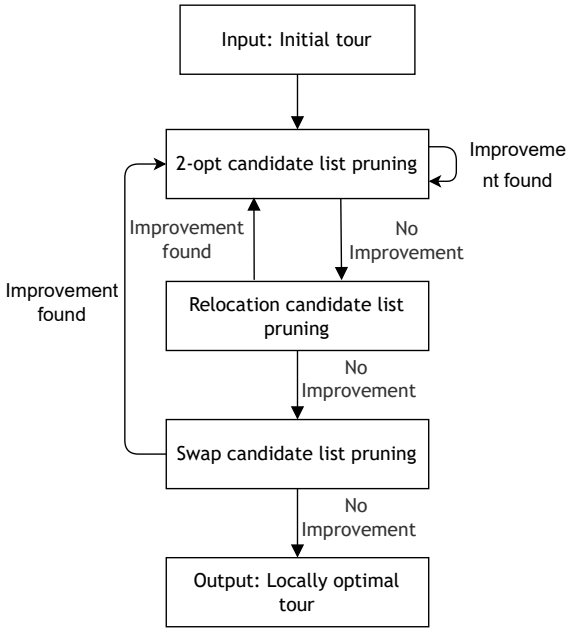


图 3. VND 邻域搜索序列流程图。

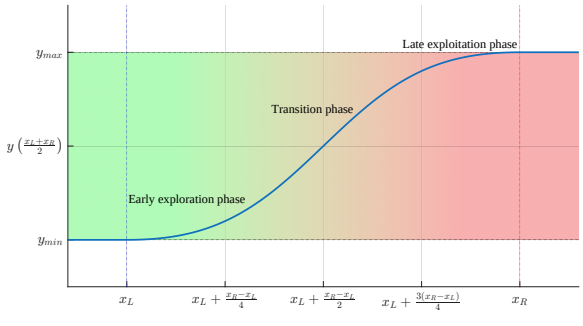


图 4. S 形 optRatio(t) 关于迭代次数的曲线图。

第一个指标是种群同质性。当蚁群收敛到较窄区域后，各蚂蚁回路代价彼此接近，可用变异系数 $\mathrm{CV} = \sigma_{\mathrm{costs}}/\mu_{\mathrm{costs}}$ 对此进行刻画，当 CV 低于预设阈值 $\varepsilon_{\mathrm{CV}}$ 时，可认为种群已趋于同质，继续搜索发现新解的可能性很低。

第二个指标是最优解停滞。即使种群仍有多样性，全局最优也可能长期停留在局部最优附近。为此，算法记录全局最优代价连续未获改进的迭代次数，在达到预设上限 $\mathrm{stagn}_{\max}$ 便终止。

为减少误停，上述两判据仅在完成 minIter 次迭代后启用，因搜索初期的代价方差较大、全局最优仍在快速改善，过早监测容易把正常波动误判为收敛。

7) 集成总结: 综合上述设计，MMAS-VND-CL 算法把每个机制组织进同一条迭代搜索流程：扩展代价矩阵 $\mathbf{D}_{\mathrm{ext}}$ 定义在含虚拟节点的 $N+1$ 个节点上，信息素模型与局部搜索均在该空间中进行。单次迭代的主要步骤如下：

1) 由 S 形曲线确定本轮接受局部搜索的蚂蚁比例；
2) 全部蚂蚁按照 ACS 风格的伪随机转移规则各自构造一条覆盖 $N+1$ 个节点的哈密顿回路；
3) 依据精英—随机混合策略选出部分蚂蚁，对其回路执行带候选列表加速的 VND 局部搜索；
4) 更新当前全局最优回路，并依据自适应停止准则判断是否终止；
5) 若继续搜索，则先蒸发信息素，再在本轮最优回路与全局最优回路上沉积，最后按 MMAS 规则对信息素取值进行动态限幅。

搜索在满足自适应停止条件时结束。终止后，从最优回路中去掉虚拟节点 V，再把剩余节点调整为起点在前、终点在后的顺序，便得到最终的开放路径访问序列。MMAS-VND-CL 算法的总体流程如图 5所示。

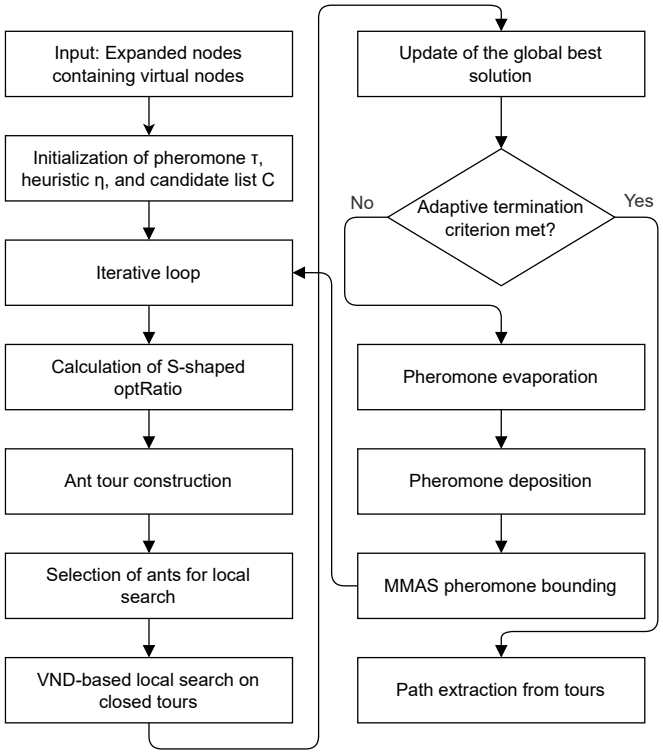


图 5. 改进蚁群优化总体流程图。

D. 协同优化框架

前述各节分别给出改进 A* 寻径层与 MMAS-VND-CL 排序层，本节将二者衔接为完整流程：先构建障碍约束代价矩阵，再在该矩阵上优化访问顺序，最后按序回取各段路径拼接为完整轨迹。

几何层完成代价矩阵构建。给定起点、终点和 K 个中间目标点（共 N 点），对每一对点求最短无碰撞路径，经安全简化与弧长参数化三次样条平滑后，以连续轨迹的欧氏弧长构成代价矩阵 $\mathbf{D} \in \mathbb{R}^{N\times N}$。$\mathbf{D}$ 并非欧氏距离矩阵，其元素对应障碍约束下的真实通行长度；以平滑弧长而非栅格步进累加计价，可削弱离散化阶梯效应，使排序依据更贴近实际行程。构建共需 $O(N^2)$ 次规划调用，工程上可用点对路径缓存避免重复搜索；若某点对无可行路径，则相应元素记为 $\mathbf{D}(i,j)=\infty$，一旦有目标点从起点不可达即中止并报告失败。

排序层完成访问顺序优化。该层仅在数值矩阵 $\mathbf{D}$ 上求解开放旅行商问题，不感知地图与障碍几何，返回最优访问顺序 $\boldsymbol{\pi}^* = (1, \pi_2, \ldots, \pi_{K+1}, N)$ 及总代价 $C(\boldsymbol{\pi}^*)$；再按该顺序从缓存中取出相邻点对的各段路径，拼成完整无障碍连续轨迹。当 $K=0$ 时，问题退化为单对最短路径查询，顺序为 $[1, N]$，代价为 $\mathbf{D}(1,N)$。

两层分离使规划器与排序求解器可独立替换与分层对照，代价矩阵 $\mathbf{D}$ 则将障碍距离拓扑压缩为紧凑接口，排序层无需接触栅格地图即可保留避障可行性；其信息损失在于无法判断两段路径是否共用狭窄走廊，但就最小化总代价而言该表示已属充分。端到端依次完成代价矩阵构建、开放旅行商求解与路径回取，系统整体架构如图 6所示。

V. 实验与分析

在给出方法设计之后，本节通过仿真对所提框架的有效性进行检验。实验从以下几个相互衔接的层面展

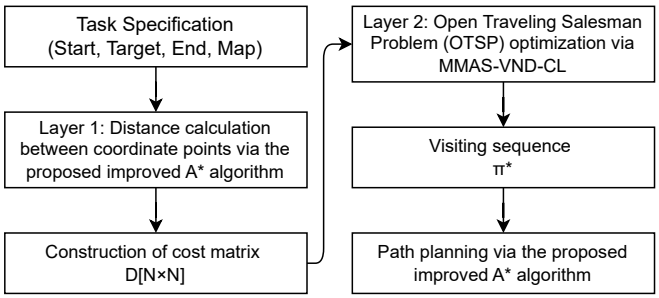


图 6. 系统架构总图

开，首先，比较基于 JPS 的改进 A* 规划器与常见基线在全局寻径上的效率与路径质量；其次，考察 MMAS-VND-CL 对关键超参数的敏感程度；然后，通过消融实验分离各组成机制的贡献；最终，将不同旅行商求解器与改进 A* 规划器组合，比较完整的多目标点遍历性能。所有对比均在统一参数策略下进行多次独立重复，以兼顾解质量与统计稳定性。

A. 实验设置

所有实验均在配备 Intel Core Ultra 5 125H 处理器与 32 GB 内存的计算机上完成，软件环境为 MATLAB R2025b。

为覆盖不同障碍形态与问题规模，本文构建了如图 7所示的两类实验场景：一类是含障碍的栅格地图（Map 1、Map 2 与 Map 3），用于检验避障寻径与多目标点遍历；另一类是标准 TSPLIB 算例 kroB100，用于单独检验大规模组合排序能力。图中绿色圆点为起点，红色三角为终点，蓝色编号点为多目标遍历实验中的中间目标点。其中 Map 1 与 Map 2 同时给出多目标遍历所需的中间目标点；在第 V-B 节全局路径规划对比实验中，仅使用各图的起点与终点进行点对点规划，不涉及中间目标点的访问顺序优化。

具体环境特征如下：

1) Map 1：50×50 栅格环境。障碍以较长的横纵墙段为主，形成若干由走廊连通的“房间”状空腔，整体近似街区或室内隔断布局，适合考察规划器在中等障碍密度下的绕行与搜索效率。
2) Map 2：80×80 栅格环境。障碍呈不规则碎块状高密度散布，自由空间被切割成大量细碎、曲折的通道，局部通行高度依赖短距离绕障，可用于检验算法在杂乱环境中的鲁棒性。
3) Map 3：80×80 迷宫环境。墙段更长且大体正交布置，构成典型的迷宫式走廊，通道狭长、转折频繁，仅用于全局路径规划对比，以突出不同寻径算法在结构化迷宫中的表现差异。
4) kroB100：TSPLIB 标准 100 城市算例。城市以平面坐标给出，重点在于大规模目标点的访问顺序优化。本文将起点与终点均取为城市 1，使开放路径退化为回路，从而可与 TSPLIB 公布的最优回路长度 22141 直接对照，用作算法在约百点规模组合搜索能力的校准基准。

多目标遍历实例的规模如下。Map 1 使用包括起点和终点在内的 50 个导航点，Map 2 使用包括起点和终点在内的 70 个导航点，起点与终点分别为各实例导航点序列的首末点，各导航点的栅格坐标见图 7的编号标注；kroB100 实例包含全部 100 个城市。

表 I
所提算法的默认参数设置

Module	Parameter	Symbol	Value
Original params	Number of ants	n_{ants}	40
	Pheromone weight	α	1
	Heuristic weight	β	2.0
	Evaporation rate	ρ	0.25
	Deposit constant	Q	225
	Initial pheromone	τ_0	0.1
ACS	Greedy probability	q_0	0.45
VND	Local search ratio	$\gamma_{\max}$	0.4
	Local search transition interval	$[x_L, x_R]$	$[0, 30]$
	Elite ratio	r_{elite}	0.7
	Candidate list size	k	9
	Local search ratio lower bound	$\gamma_{\min}$	0.0
	S-curve steepness	a	2.0
Adaptive stopping	Homogeneity threshold	ε_{CV}	10^{-3}
	Stagnation limit	$\text{stagn}_{\max}$	70
	Minimum iterations	minIter	30
	Maximum iterations	n_{iter}	800

关于重复次数与统计口径，全局路径规划对比中每个算法在每张地图上运行 10 次并取时间平均值；旅行商相关实验中每种配置运行 40 次，报告最优代价值、平均收敛代价及标准差、运行时间、成功率统计量。旅行商相关实验的运行时间为在预计算代价矩阵上的求解器耗时，不含 $O(N^2)$ 代价矩阵构建与路径后处理的开销。

参数设置方面，对于本文提出的 MMAS-VND-CL 算法的主要参数，全文使用表 I中的默认参数值。

评估指标包括路径长度、计算时间、扩展节点数（仅限全局规划）、最优代价、平均收敛代价、成功率和标准差。其中参考最优代价的来源分两类。kroB100 的参考值 22141 为 TSPLIB 公布的最优回路长度；Map 1 与 Map 2 的参考值分别为 352.56 与 623.78，是本次实验全部算法全部运行中的最好观测值。成功率定义为达

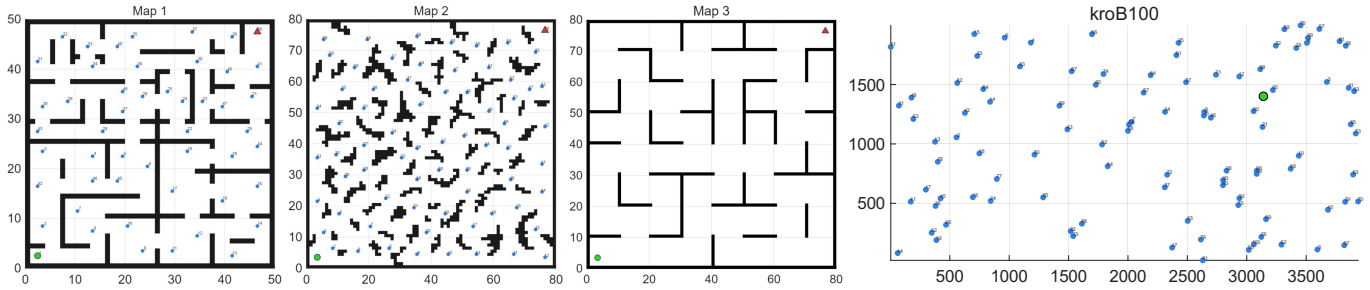


图 7. 实验环境示意图。Map 1 与 Map 2 中，绿色圆点为起点，红色三角为终点，蓝色编号点为多目标遍历的中间目标点；Map 3 仅给出起终点；kroB100 为 100 城市平面分布，绿色圆点表示开放路径的端点。

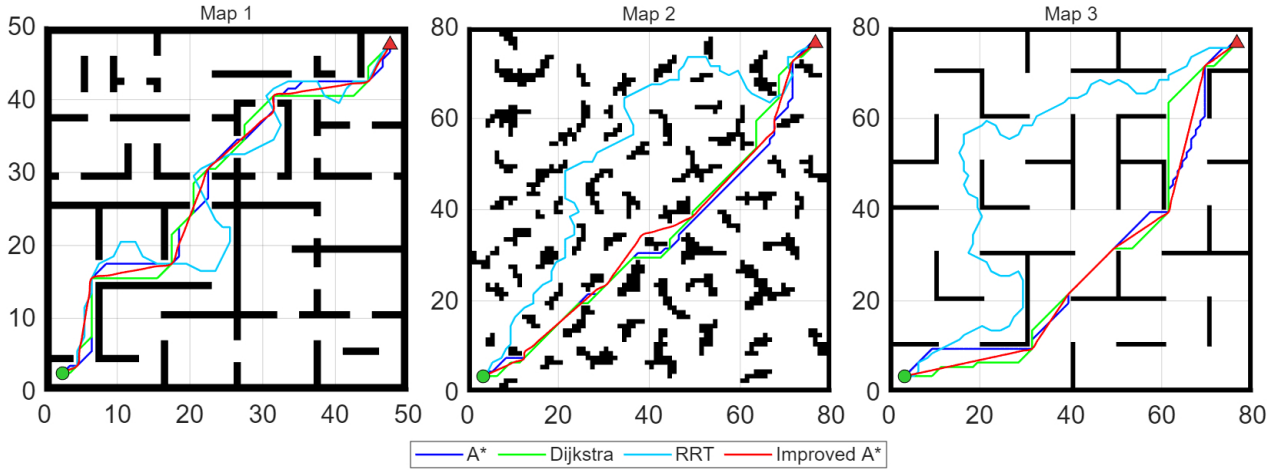


图 8. 四种算法在三张地图上的路径规划结果。

到参考最优代价的运行占比。由于 Map 1 与 Map 2 的参考值来自观测而非认证最优，基线成功率 0% 应理解为未达到本次实验的最好已知值，而非无可行解。

B. 全局路径规划对比

本组实验对比所提基于 JPS 的改进 A* 规划器与传统 A*、Dijkstra 及 RRT 三种基线，从路径长度、计算时间与扩展节点数三个维度检验寻径质量与搜索效率。每个规划器均在 Map 1、Map 2 与 Map 3 上独立运行 10 次并取时间平均，结果汇总于表 II，路径可视化与长度、时间对比分别见图 8–10。总体而言，所提方法在三张地图上同时取得最短路径、更少的节点扩展与更短耗时；采样类方法可能在个别场景中以牺牲路径质量换取极短耗时，而碎块障碍会削弱 JPS 相对传统 A* 的剪枝幅度。下面结合指标、条件与算法机制展开说明。

首先看路径长度。表 II显示，所提方法在 Map 1–Map 3 上分别达到 72.93、108.17 与 113.33，较传统 A* 与 Dijkstra 依次缩短约 4.7%、2.9% 与 5.3%；图 9的折线趋势与表格一致，表明该优势并非单点偶然。需要说明的是，传统 A* 与 Dijkstra 在一致启发下给出的是栅格度量下的最优折线，二者长度相同正说明它们都触及了离散最优；所提方法更短，主要并非“搜到另一条更短的栅格路径”，而是后处理把阶梯状折线变成了更紧凑的连续轨迹——安全感知简化合并冗余航点、削减绕角，样条平滑进一步消除锯齿，从而更接近物理上的连续最短行程。换言之，长度优势与第 IV-B 节的简化—平滑机制直接对应。

表 II
全局路径规划性能对比

Algorithm	Map	Path Length	Time (ms)	Expanded Nodes
Improved A*	Map 1	72.93	2.45	70
	Map 2	108.17	11.85	415
	Map 3	113.33	4.31	48
A*	Map 1	76.53	6.14	811
	Map 2	111.44	29.19	1979
	Map 3	119.64	17.88	2796
Dijkstra	Map 1	76.53	24.26	1997
	Map 2	111.44	202.29	5217
	Map 3	119.64	204.46	5664
RRT	Map 1	100.80	7.42	—
	Map 2	135.40	19.33	—
	Map 3	144.07	3.69	—

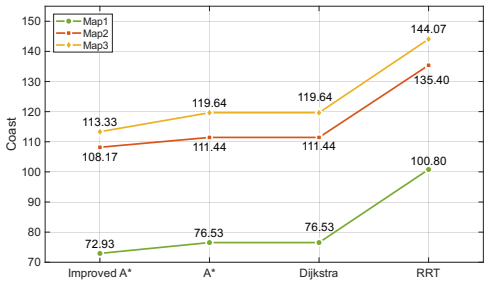


图 9. 四个算法在 Map 1、2、3 上的路径长度对比。

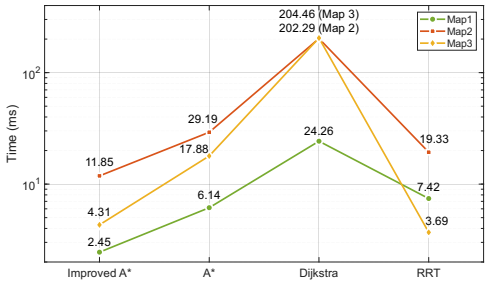


图 10. 四个算法在 Map 1、2、3 上的计算时间对比。

其次看搜索效率。扩展节点数上，所提方法在三张图上均远低于两个图搜索基线：相对 A* 分别减少约 91.4%、79.0% 与 98.3%，相对 Dijkstra 则均减少约 92% 以上。这一现象可由 JPS 的对称性剪枝解释：搜索只需在被迫邻居与方向变化处生成跳点，中间同向栅格不再逐一入堆，搜索树因而被大幅压缩；在同一地图上 Dijkstra 向各方向均匀扩展、传统 A* 虽有启发引导但仍在逐格松弛，故扩展数呈数量级差距。同时也应客观看到，Map 2 上相对 A* 的降幅明显小于 Map 1 与 Map 3：障碍碎块使被迫邻居更频繁，跳点数量上升，剪枝收益被部分抵消，但整体上仍保持对逐格展开方法的数量级优势。

再与采样类方法比较。RRT 在 Map 3 上以 3.69 ms 成为耗时最短者，这一结果需要如实指出：当迷宫通道规则、起终点近乎直线连通时，随机树可能较快找到一条可行解。但代价是路径质量显著下降——RRT 长度在三张图上均最长，较所提方法高约 25%–38%，且作为随机采样方法其结果存在运行间波动，与采样一致性较差的典型特征相符。因而在“既要短路径、又要稳定可重复”的评价口径下，RRT 并不构成对所提方法的实质威胁；若任务仅要求快速得到任意可行解，则采样法仍有其适用空间。

综上，三个指标可以相互印证：长度优势来自简化—平滑对连续行程的逼近，扩展与耗时优势来自 JPS 对搜索树的剪枝，二者分别对应后处理管线与图搜索内核的不同机制。Map 2 上剪枝幅度相对收窄提示，在高度杂乱的栅格中被迫邻居会削弱 JPS 的对称性收益，但扩展数与耗时仍全面优于逐格展开的图搜索。后续工作可考虑在高杂乱地图上自适应调整剪枝或跳点生成策略，以进一步压低此类场景的搜索开销。

C. 参数敏感性分析

本组实验考察 MMAS-VND-CL 对五个关键超参数的敏感性，即蚂蚁数量 n_{ants}、贪心选择概率 q_0、后期局部搜索比例 $y_{\max}$、精英比率 r_{elite} 与候选列表大小 k。本实验首先对这 5 个参数均设定了五个参数等级，如表 III所示，后续实验组的取值水平用 $n_{\text{ants}}/q_0/y_{\max}/r_{\text{elite}}/k$ 的组合方式表示。实验中采用单因素变动方式，每次只将一个参数在五个水平上取值，其余参数保持默认，共实施 21 组单因素配置，其中，3/3/3/4/2 实验组为默认参数水平组合；每组在 Map 1、Map 2 与 kroB100 上各独立运行 40 次。评价不只看最优代价，还同时观察平均代价/标准差、成功率与平均收敛时间，各配置的完整数值已整理至配套实验数据文档中的敏感性结果表，正文不再逐一罗列；代价—时间趋势见图 11，默认参数下的路径形态见图 12。

对于 Map 1、Map 2 和 kroB100，参考最优代价分别为 352.56、623.78 和 22141。从参考最优代价看，配套实验数据文档中的敏感性结果显示，全部配置都能触及同一参考值，说明 MMAS-VND-CL 在合理参数范围内具备较强的“找到优解”能力；真正拉开差距的是成功率、平均代价与耗时。结合图 11可以得到的主要结论是：q_0 与 $y_{\max}$ 最为敏感（尤其是 q_0 取大、$y_{\max}$ 取小时退化明显），n_{ants} 与 r_{elite} 居中，k 在适中范围内较平缓但过小会崩溃；默认组合（3/3/3/4/2）在质量、稳定性与时间之间较为均衡，因此被后续实验固定采用。下面按参数分别说明。

从参数规律看，q_0 与 $y_{\max}$ 最为敏感，n_{ants} 与 r_{elite} 居中，k 在适中范围内较平缓但过小会崩溃。q_0 控制伪随机转移中利用与探索的配比，取 0.90（近纯贪心）时蚁群几乎总是锁定当前得分最高的候选，信息素迅速坍缩到少数边上，kroB100 成功率跌至 57.5%、标准差增至 45.78，Map 2 也只有 70%；降至 0.10–0.30 虽可保持 100% 成功率，耗时却明显更长（约 490–720 ms）。$y_{\max}$ 决定后期 VND 精化力度，取 0.10 时缺少后期加密局部搜索，Map 2 成功率仅 62.5%；取 0.90 时成功

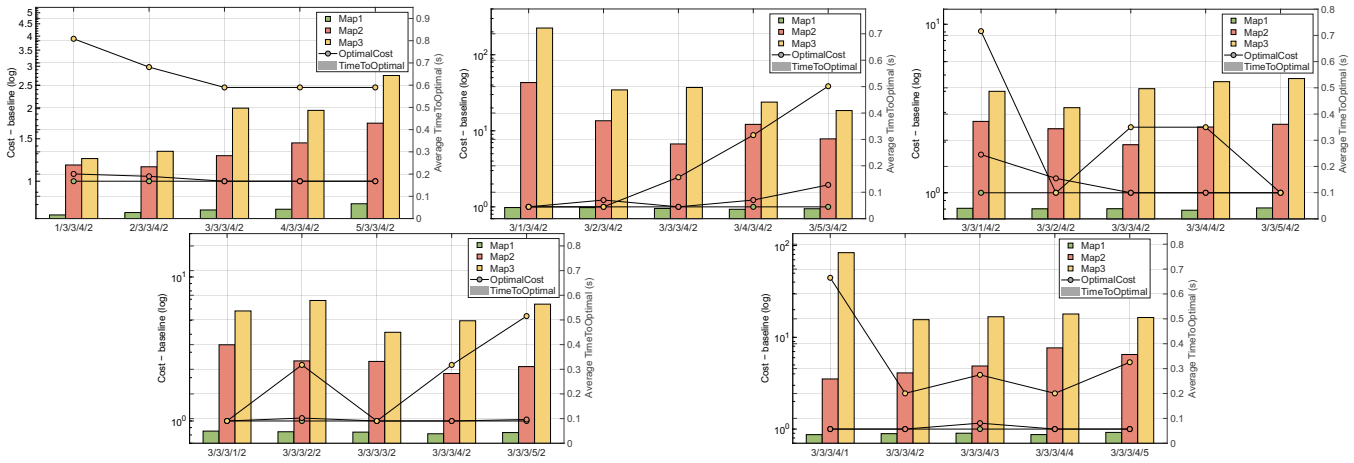


图 11. 各参数水平在 Map 1、Map 2、kroB100 上的代价与计算时间对比。

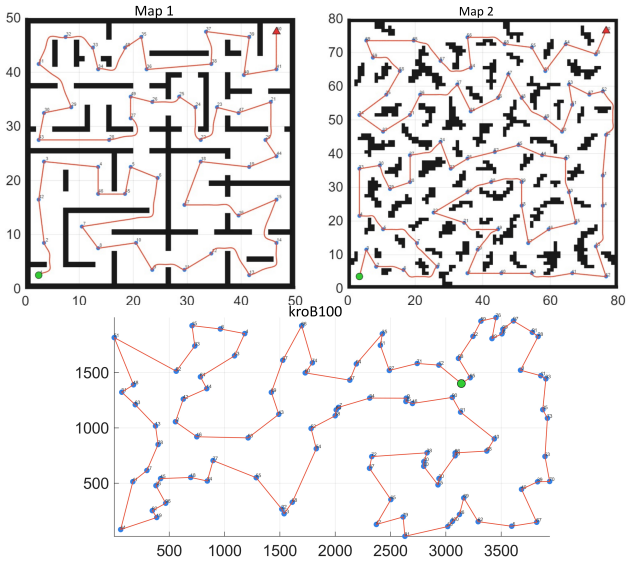


图 12. 默认参数设置下算法在 Map 1、Map 2、kroB100 上规划的路径图。

表 III
敏感性分析中使用的参数水平

Parameter	Parameter Levels				
	Level 1	Level 2	Level 3	Level 4	Level 5
n_{ants}	10	20	**40**	60	90
q_0	0.10	0.30	**0.45**	0.60	0.90
$y_{\max}$	0.10	0.20	**0.40**	0.60	0.90
r_{elite}	0.1	0.3	0.5	**0.7**	0.9
k	3	**9**	15	30	60

率反而回落，推测过强局部搜索在部分运行中过早锁死到次优结构。

n_{ants} 从 10 增至 40 使 Map 2 成功率由 92.5% 升至 100%、kroB100 由 95% 升至 97.5%，种群过小时信息素更新易被少数个体带偏、方差增大；继续增至 60–90 成功率不再提升而耗时提高约 20%–60%，属“够用且不贵”的折中。r_{elite} 取 0.1–0.5 时 kroB100 成功率可达 100% 或接近 100%，0.9 过度精英化反而降至 92.5%。k 在 9–60 范围内对代价影响不大，但 $k = 3$ 时候选剪枝把有希望的边交换一并滤掉，成功率降至 42.5%、耗时增至约 765.9 ms。

还应指出，并不存在在所有指标上唯一占优的配置，评价须在命中质量、稳定性与运行时间之间权衡。默认组合 40/0.45/0.40/0.7/9 在 Map 1、Map 2 上实现 100% 命中与零方差，在 kroB100 上以约 500 ms 达到 97.5% 成功率与较小标准差，综合表现最均衡，故后续消融与算法对比实验均沿用该组设定。

D. 蚁群优化组件消融实验

为分离 MMAS-VND-CL 各机制的独立贡献，本组实验在保持其余设置不变的前提下，每次仅关闭一个机制，共设置六组，如表 IV所示。每组在 Map 1、Map 2 与 kroB100 上运行 40 次，仍以最优代价、平均代价/标准差与耗时作为联合评价，实验的总结果见表 V，各组的平均代价与平均计算时间对比如图 13。总体结论是：VND 与 MMAS 边界共同支撑“能否稳定拿到好解”，二者缺一不可；伪随机转移、S 形调度与候选列表则更偏向“多快拿到好解”，对解质量的边际影响相对有限，但决定着时间开销。需要提前说明的是，部分消融组的平均代价仍能接近完整算法，甚至耗时更短，这类“看似更好”的现象并不与上述层次矛盾，下面结合机制展开。

首先看 VND（组 3）。关闭局部搜索后解的质量明显变差：三张地图平均代价均上升，Map 1 由 352.56

表 IV
消融实验分组

Group	Description	Disabled Mechanism
1	Complete Algorithm	None
2	No Pseudo-Random	ACS Pseudo-Random Transfer Rule
3	No VND	VND Local Search
4	No S-Curve	Progressive Local Search Scheduling
5	No Candidate List	Candidate List Acceleration
6	No MMAS	MMAS Pheromone Boundary

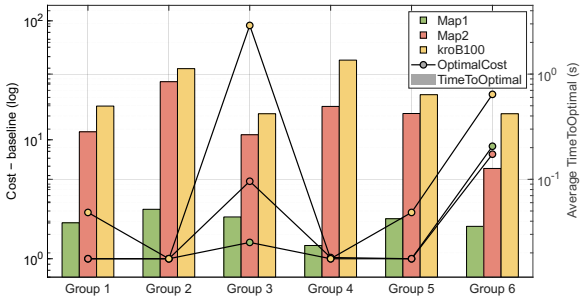


图 13. 六个消融组在三张地图上的平均代价与平均计算时间对比。

表 V
消融实验结果

Map	Group	Best cost	Path Mean/std.dev	Run Time (ms)	Success rate (%)
Map 1	1	**352.56**	**352.56/0.00**	38.6	**100**
	2	**352.56**	**352.56/0.00**	51.9	**100**
	3	**352.56**	352.93/0.75	43.9	80
	4	**352.56**	**352.56/0.00**	**23.5**	**100**
	5	**352.56**	**352.56/0.00**	42.2	**100**
	6	**352.56**	360.39/13.81	35.7	52.5
Map 2	1	**623.78**	**623.78/0.00**	282.9	**100**
	2	**623.78**	**623.78/0.00**	846.4	**100**
	3	**623.78**	627.26/2.53	265.2	10
	4	**623.78**	623.80/0.15	492.9	97.5
	5	**623.78**	**623.78/0.00**	422.2	**100**
	6	**623.78**	630.34/6.47	**126.8**	10
kroB100	1	**22141.00**	22142.45/9.17	496.7	97.5
	2	**22141.00**	**22141.00/0.00**	1126.3	**100**
	3	**22141.00**	22231.50/73.36	**419.9**	12.5
	4	**22141.00**	22141.00/0	1357.6	**100**
	5	**22141.00**	22142.45/9.17	637.1	97.5
	6	**22141.00**	22164.10/41.74	**419.9**	72.5

升至 352.93，Map 2 由 623.78 升至 627.26，kroB100 由 22142.45 升至 22231.50；标准差同步扩大，kroB100 由 9.17 增至 73.36，运行间稳定性明显下降。这与机制预期一致，构建阶段加信息素只能给出还不错的顺序，VND 才是把回路压向局部乃至全局最优的关键精化步骤；没有它，种群只能在较粗的解附近徘徊，平均代价与方差同步上升。

其次是 MMAS 信息素边界（组 6）。去掉 $\tau_{\min}/\tau_{\max}$ 动态限幅后，三张地图平均代价均明显变差：Map 1 由 352.56 升至 360.39，Map 2 由 623.78 升至 630.34，kroB100 由 22142.45 升至 22164.10；标准差同步扩大，Map 1 达 13.81、kroB100 达 41.74。原因是失去边界约束后，早期优势边上的信息素可被无限放大，搜索很快锁死在个别路径结构上，出现典型早熟；多次运行落入不同次优盆地，故平均代价系统性偏高且波动较大。值得注意的例外是：组 6 在三张地图上耗时都不高，Map 2 上甚至最短。这并不表示算法更快，而是种群过早同质化后，自适应停止准则更早触发、无效迭代变少——以解质量换来了表面上的速度。

再看两个偏效率的组件。去掉伪随机转移（组 2）后，三张地图平均代价与完整算法持平（Map 1 与 Map 2 均为参考最优且零方差，kroB100 平均 22141.00），但耗时接近完整算法的 2–3 倍，说明受控贪心的价值在于加速收敛、节省搜索预算，与第 V-C 节“更低 q_0 更稳却更慢”的观察相互印证。去掉 S 形调度（组 4）即从迭代起就按上限实施局部搜索，kroB100 平均代价仍达 22141.00、Map 2 仅微升至 623.80，但耗时增加约 1.7 倍，Map 1 上耗时反而最短，说明固定高强度局部搜索在小规模未必亏损，但在中大型实例上会迅速耗尽计算预算；S 形调度把精化推迟到迭代后半程，正是为了在早期多探索与后期再精修之间取得平衡。去掉候选列表（组 5）平均代价与完整算法一致，耗时上升，符合其不改变邻域最优性、只减少无效评估的定位。

综上，VND 决定能否稳定逼近最优，MMAS 边界决定种群会不会过早锁死，二者构成解质量的两大支柱；伪随机转移、S 形调度与候选列表主要调节收敛速度与计算开销。完整算法在 Map 1 与 Map 2 上平均代价均达到参考最优且零方差，kroB100 上平均代价 22142.45、标准差 9.17，以约 500 ms 运行，是质量、稳定性与时间的均衡点，后续算法对比实验均以组 1 的完整配置为准；若应用场景更看重解质量的绝对水平，可加强后期局部搜索比例或延长停滞上限。

E. 不同旅行商求解器的多目标点遍历对比

在固定前端改进 A* 代价矩阵与虚拟节点开放编码的前提下，本组实验更换后端旅行商求解器，以检验完整多目标点遍历框架中“排序模块”的实际贡献。参与对比的四种求解器为：本文 MMAS-VND-CL、标准蚁群系统（ACO）、遗传算法（GA）与模拟退火（SA），参数见表 VI；每种组合在 Map 1、Map 2 与 kroB100 上独立运行 40 次。结论可以概括为：在统一几何代价接口与所测参数配置下，MMAS-VND-CL 在解质量、成

表 VI
对比旅行商求解器的参数设置

Algorithm	Main parameters
MMAS-VND-CL	$n_{\text{ants}} = 40$; $\alpha = 1$; $\beta = 2$; $\rho = 0.25$; $Q = 225$; $\tau_0 = 0.1$; $q_0 = 0.45$; $y_{\max} = 0.4$; $r_{\text{elite}} = 0.7$; $k = 9$ Local search transition interval= $[0, 30]$;
ACO	$n_{\text{ants}} = 40$; $\alpha = 1$; $\beta = 2$; $\rho = 0.25$; $Q = 225$; $\tau_0 = 0.1$; Iter = 100
GA	Pop = 100; Iter = 1500; $P_c = 0.8$; $P_m = 0.25$; Number of elites= 10
SA	$T_0 = 150$; $\alpha = 0.996$; $L = 10$; $T_{\text{end}} = 10^{-8}$

表 VII
多目标点遍历对比结果

Map	Algorithm	Best cost	Path Mean/std.dev	Run Time (ms)	Success rate (%)
Map 1	MMAS-VND-CL	**352.56**	**352.56/0.00**	**38.6**	**100**
	ACO	361.41	391.53/18.44	90.6	0
	GA	411.07	496.28/46.93	4181.1	0
	SA	365.13	413.82/25.83	784.6	0
Map 2	MMAS-VND-CL	**623.78**	**623.78/0.00**	241.5	**100**
	ACO	641.09	691.64/18.61	**223.2**	0
	GA	710.83	868.79/66.45	6693.6	0
	SA	655.71	687.25/13.02	506.3	0
kroB100	MMAS-VND-CL	**22141.00**	**22142.45/9.17**	**496.7**	**97.5**
	ACO	23153.00	24546.05/606.87	534.1	0
	GA	32243.00	36772.55/2894.73	8817.2	0
	SA	23487.00	24871.05/753.20	516.5	0

功率与稳定性上均优于所对比的三种基础元启发式实现，多数场景下时间也处于较低水平；仅 Map 2 上轻量 ACO 略快，但代价质量差距明显，GA 则在质量与时间上均明显落后。下面按指标—条件—机制展开，并如实讨论个别“更快却不更好”的情形。

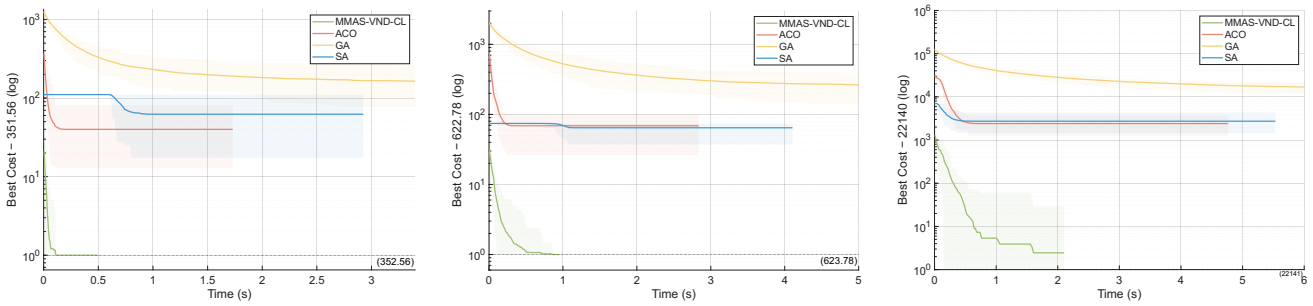
每个求解器的多目标点遍历性能见表 VII。对应可视化结果见图 14–17。

首先考察解的质量与成功率。表 VII显示，MMAS-VND-CL 在三张图上均取得最低的最优代价和平均代价，并且是唯一能在多次运行中稳定达到参考最优代价的求解器：Map 1 与 Map 2 上 40 次运行全部达到参考值 352.56 与 623.78，kroB100 上以 97.5% 的成功率达到参考值 22141、平均 22142.45。其余三个对比算法的成功率均为 0%，即 40 次运行均未达到参考最优代价；这并不表示其无可行解，表中可见其平均代价介于 361.41 与 36772.55 之间。差距在 kroB100 上最为明显：所对比的 ACO 实现平均代价 24546.05 较本文高约 10.9%，SA 高约 12.3%，GA 平均 36772.55 则高出约 66%。图 17中基线路径的迂回与交叉明显多于本文方法，图 15的箱体也更长、须更散，说明其“偶尔并不太差、整体却系统性偏长”。

其次分析算法的稳定性。MMAS-VND-CL 在 kroB100 上标准差仅 9.17，而 ACO、SA、GA 分别为 606.87、753.20 与 2894.73，较 ACO 与 SA 小约 1.8 至 1.9 个数量级、较 GA 小约 2.5 个数量级；Map 2 上 SA 虽已收敛到 13.02，仍高于本文的零方差。结合机制可以理解：虚拟节点编码使起终点固定且完整参与信息素更新，MMAS 边界避免早期锁死，候选列表加速的 VND 则在每一代对精英回路做强力精化，三者叠加把运行之间的波动压到很低；缺少 VND 与边界控制的标准 ACO、SA 只能依赖构建/邻域的随机扰动，路径质量随初始化散布较宽。

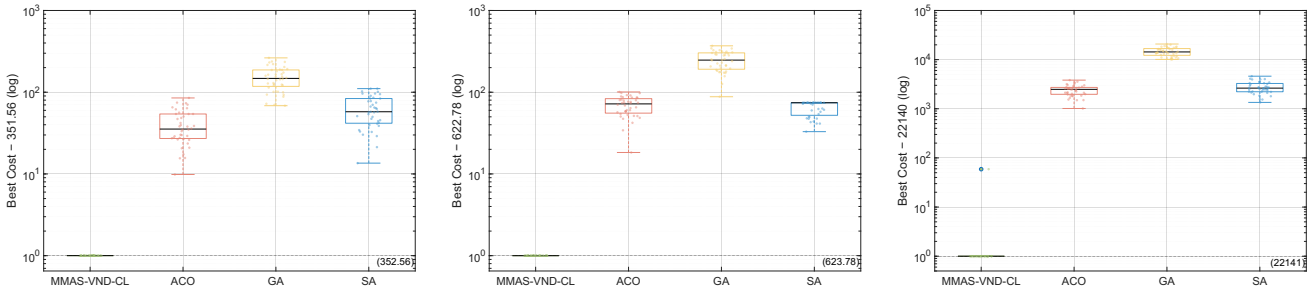
随后讨论时间性能，并对其作客观评价与必要让步。表 VII中，Map 2 上 ACO 以 223.2 ms 略快于本文的 241.5 ms，是唯一在耗时上占优的基线。原因很直接：该基线没有大规模 VND 邻域搜索，迭代一次更便宜；本文把部分预算花在局部搜索上，单次迭代更重。但在 Map 1（38.6 ms vs. ACO 90.6 ms）与 kroB100（496.7 ms，快于 SA 的 516.5 ms 与 ACO 的 534.1 ms）上，本文方法反而更快——候选列表剪枝使 VND 的额外开销被压缩在可控范围内，而基线为达到可比质量往往需要更多迭代。GA 则全面落后，三图耗时达数秒，代价仍最差，说明其变长染色体交叉在路径型表示上缺乏针对性，搜索浪费明显。综合图 16的“时间—代价”折中：本文方法同时占据低代价与较低耗时的一侧，Map 2 上以可忽略的时间差换取了显著更优的解质量。

最后作机制归因与收束。在统一的 JPS 代价矩阵上，组合层的差别被单独放大：虚拟节点开放编码保证首尾约束不被搜索破坏；MMAS 信息素边界维持多样性；ACS 式伪随机加速收敛；候选列表 VND 负责把回路压向近优；S 形调度与自适应停止则控制何时、为谁支付精化成本。实验表明，真正拉开与基线差距的是“稳定达到参考最优代价”这一能力，而不仅是某一次的最好值。同时需要说明对比条件的边界：基线采用单组参数与固定迭代预算，未进行与本文方法同等的参数搜索；本文方法的默认参数经敏感性分析选定，迭代上限为 800 代并叠加 VND 局部搜索。当然也应承认：若应用只要求快速得到任意可行访问序，轻量 ACO 在简单场景上仍有低延迟价值；若目标点规模继续增大，VND 的计算占比会上升，届时需要进一步加强候选列表压缩或分层局部搜索。综合质量、可靠性与计算开销，在所测配置下 MMAS-VND-CL 更适合作为障碍环境多目标点遍历的排序模块。



(a) Map 1 (b) Map 2 (c) kroB100

图 14. 四个旅行商求解器在三张地图上 40 次运行的平均时间-代价收敛曲线。



(a) Map 1 (b) Map 2 (c) kroB100

图 15. 四个旅行商求解器在三张地图上 40 次运行的代价分布。

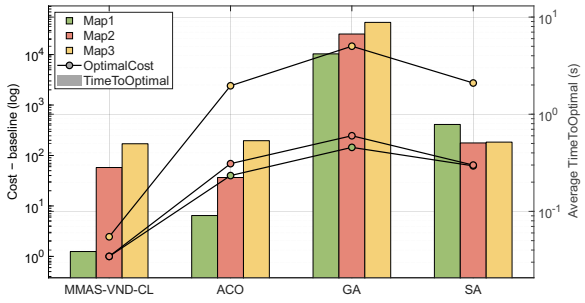


图 16. 四个旅行商求解器在三张地图上的平均代价与平均计算时间对比。

除上述仿真地图外，本文还在一块由卫星影像构建的栅格地图上做了定性演示。从 Google 地图截取该区域后，将其转换为二进制栅格（黑色为障碍物、白色为可通行空地），并对转换后的地图做了适度手工优化以贴近导航需求。图 18自左至右依次为：原始卫星图（红框为实验场地）、二值化地图与目标点配置（黄点为中间目标，绿点为起点，红三角为终点），以及在真实地图背景上显示的规划结果。演示中算法在该地图上生成了穿越可通行区域、合理连接各目标点的连续路径。需要说明，该演示为单张二值化地图上的定性展示，未包含量化指标、重复统计或实物运行，其结论限于方法可直接处理由真实影像构建的栅格环境这一点。

VI. 结论

本文提出一种用于静态障碍环境多目标点遍历的两层规划框架，以基于 JPS 的改进 A* 搜索点对路径，结合安全距离感知的路径简化和样条平滑构建障碍约束代价矩阵，并由 MMAS-VND-CL 确定目标访问顺序。实验表明，改进 A* 在 Map 2 中的规划时间为 11.85 ms，低于传统 A*（29.19 ms）及 Dijkstra（202.29 ms）；处理后连续轨迹较所比较的原始栅格路径缩短约 4.7%–5.3%。在 kroB100 标准闭环基准上，MMAS-VND-CL 以 97.5% 的命中率达到已知最优回路代价 22141，平均代价为 22142.45。消融结果显示，VND 局部搜索与 MMAS 信息素边界对求解质量和稳定性具有重要作用。基于卫星影像构建的栅格地图也展示了该方法处理实际场景障碍布局的能力。

参考文献

[1] C. Chen, J. Frey, P. Arm, and M. Hutter, “SMUG planner: A safe multi-goal planner for mobile robots in challenging environments,” *IEEE Robot. Autom. Lett.*, vol. 8, no. 11, pp. 7170–7177, 2023, doi: 10.1109/LRA.2023.3311207.

[2] T. T. C. Giang, H. T. T. Binh, H. V. D. Luong, N. H. Hoang, and D. Q. Huy, “Fast marching firework method for multi-goal mobile robot path planning in complex obstacle maps,” *Intell. Service*

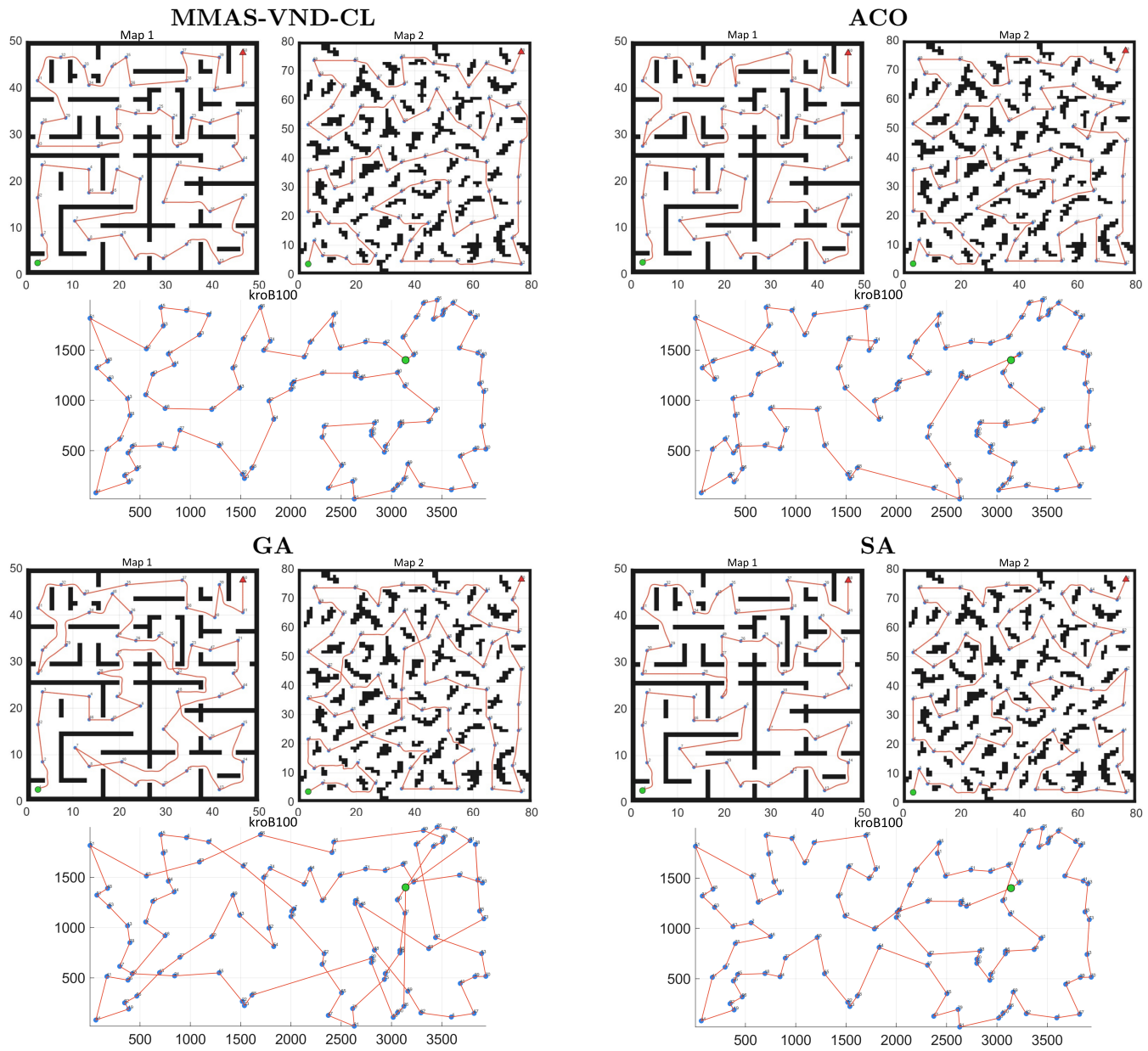


图 17. 各旅行商求解器在 Map 1、Map 2、kroB100 上规划的最优代价路径图。

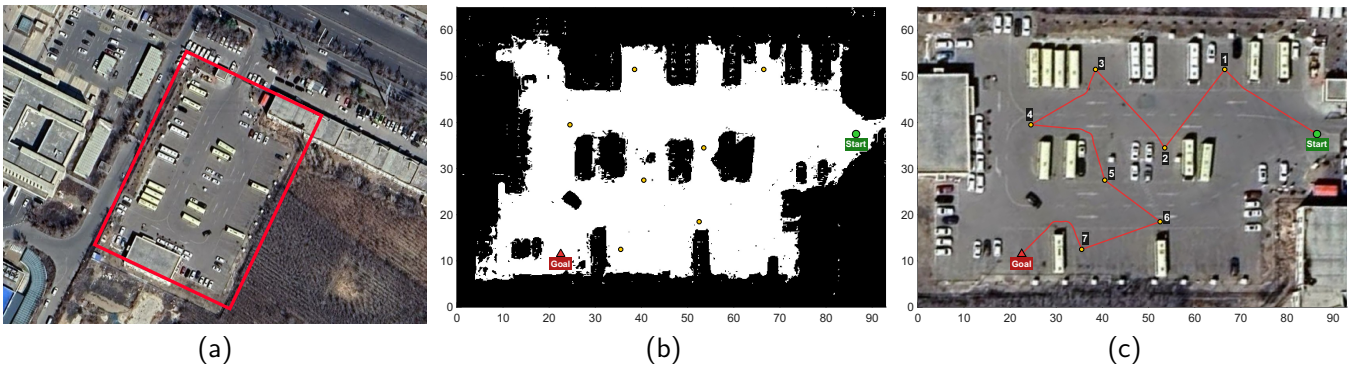


图 18. 真实卫星场景验证。(a)Google 地图截取区域，红框为实验场地; (b) 转换后的二进制地图与目标点配置; (c) 真实地图背景上的规划路径。

Robot., vol. 18, no. 6, pp. 1467–1484, 2025, doi: 10.1007/s11370-025-00654-6.

[3] K. Karur, N. Sharma, C. Dharmatti, and J. E. Siegel, "A survey of path planning algorithms for mobile robots," *Vehicles*, vol. 3, no. 3,

pp. 448–468, 2021, doi: 10.3390/vehicles3030027.

[4] J. Wang and M. Q.-H. Meng, "Real-time decision making and path planning for robotic autonomous luggage trolley collection at airports," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 52, no. 4, pp. 2174–2183, 2022, doi: 10.1109/TSMC.2020.3048984.

[5] A. Bolu and Ö. Korçak, "Adaptive task planning for multi-robot smart warehouse," *IEEE Access*, vol. 9, pp. 27346–27358, 2021, doi: 10.1109/ACCESS.2021.3058190.

[6] S. Saller, J. Koehler, and A. Karrenbauer, "A survey on approximability of traveling salesman problems using the TSP-T3CO definition scheme," *Ann. Oper. Res.*, vol. 351, no. 3, pp. 2129–2190, 2025, doi: 10.1007/s10479-025-06641-5.

[7] P. C. Pop, O. Cosma, C. Sabo, and C. Pop Sitar, "A comprehensive survey on the generalized traveling salesman problem," *Eur. J. Oper. Res.*, vol. 314, no. 3, pp. 819–835, 2023, doi: 10.1016/j.ejor.2023.07.022.

[8] J. Faigl, "On the performance of self-organizing maps for the non-Euclidean traveling salesman problem in the polygonal domain," *Inf. Sci.*, vol. 181, no. 19, pp. 4214–4229, 2011, doi: 10.1016/j.ins.2011.05.019.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[10] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959.

[11] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Dept. Comput. Sci., Iowa State Univ., Ames, IA, USA, Tech. Rep. 98-11, 1998.

[12] J. A. Abdulsaheb and D. J. Kadhim, "Classical and heuristic approaches for mobile robot path planning: A survey," *Robotics*, vol. 12, no. 4, p. 93, 2023, doi: 10.3390/robotics12040093.

[13] K. Katona, H. A. Neamah, and P. Köröndi, "Obstacle avoidance and path planning methods for autonomous navigation of mobile robot," *Sensors*, vol. 24, no. 11, p. 3573, 2024, doi: 10.3390/s24113573.

[14] D. T. Xuan, N. T. Hung, and V. T. Thang, "A comprehensive review of improved A* path planning algorithms and their hybrid integrations," *Automation*, vol. 6, no. 4, p. 52, 2025, doi: 10.3390/automation6040052.

[15] M. Likhachev, G. Gordon, and S. Thrun, "ARA*: Anytime A* with provable bounds on sub-optimality," in *Adv. Neural Inf. Process. Syst. (NIPS)*, 2003, pp. 767–774.

[16] D. Harabor and A. Grastien, "Online graph pruning for pathfinding on grid maps," in *Proc. 25th AAAI Conf. Artif. Intell. (AAAI-11)*, 2011, pp. 1114–1119.

[17] D. Harabor and A. Grastien, "Improving jump point search," in *Proc. 24th Int. Conf. Automated Planning Scheduling (ICAPS-14)*, 2014, pp. 128–135.

[18] D. Harabor and A. Grastien, "The JPS pathfinding system," in *Proc. Int. Symp. Comb. Search (SoCS)*, vol. 3, no. 1, pp. 207–208, 2021, doi: 10.1609/socs.v3i1.18254.

[19] C. Gao, H. Ding, D. Zhang, and Y. Wu, "Improved JPS and DWA path planning algorithm for mobile robots," *Mechatronics*, vol. 118, p. 103540, 2026, doi: 10.1016/j.mechatronics.2026.103540.

[20] R. Guo, X. Quan, and C. Bao, "Research on mobile robot path planning based on an improved bidirectional jump point search algorithm," *Electronics*, vol. 14, no. 8, p. 1669, 2025, doi: 10.3390/electronics14081669.

[21] "Adaptive weight optimization based jump point Theta* algorithm in mobile robot path planning for intricate environments," *Discover Appl. Sci.*, vol. 7, p. 11, 2025, doi: 10.1007/s42452-025-07932-z.

[22] I. Munasinghe, A. G. Perera, S. Anavatti, and M. Garratt, "UGV path optimization in UAV-assisted environments using visibility-aware path simplification," *J. Sens. Actuator Netw.*, vol. 15, no. 3, p. 41, 2026, doi: 10.3390/jsan15030041.

[23] J. Ahmad and M. N. Ab Wahab, "Enhancing the safety and smoothness of path planning through an integration of Dijkstra's algorithm and piecewise cubic Bezier optimization," *Expert Syst. Appl.*, vol. 289, p. 128315, 2025, doi: 10.1016/j.eswa.2025.128315.

[24] O. E. Turan, O. Misir, and M. Ozden, "Adaptive A*/NSGA-II framework for multi-goal navigation of mobile robots," *Meas. Sci. Technol.*, 2026, doi: 10.1088/1361-6501/ae9209.

[25] F. Huo, S. Zhu, H. Dong, and W. Ren, "A new approach to smooth path planning of Ackerman mobile robot based on improved ACO algorithm and B-spline curve," *Robot. Auton. Syst.*, vol. 175, p. 104655, 2024, doi: 10.1016/j.robot.2024.104655.

[26] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: Univ. Michigan Press, 1975.

[27] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, 1996.

[28] Y. Wang and Z. Han, "Ant colony optimization for traveling salesman problem based on parameters optimization," *Appl. Soft Comput.*, vol. 107, p. 107439, 2021, doi: 10.1016/j.asoc.2021.107439.

[29] M. Dorigo and L. M. Gambardella, "Ant colony system: A cooperative learning approach to the traveling salesman problem," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 53–66, 1997.

[30] T. Stützle and H. H. Hoos, "MAX–MIN ant system," *Future Gener. Comput. Syst.*, vol. 16, no. 8, pp. 889–914, 2000.

[31] P. Hansen and N. Mladenović, "Variable neighborhood search: Principles and applications," *Eur. J. Oper. Res.*, vol. 130, no. 3, pp. 449–467, 2001.

[32] N. A. Kyriakakis, M. Marinaki, and Y. Marinakis, "A hybrid ant colony optimization–variable neighborhood descent approach for the cumulative capacitated vehicle routing problem," *Comput. Oper. Res.*, vol. 134, p. 105397, 2021.

[33] X. Zhou, H. Ma, J. Gu, H. Chen, and W. Deng, "Parameter adaptation-based ant colony optimization with dynamic hybrid mechanism," *Eng. Appl. Artif. Intell.*, vol. 114, p. 105139, 2022, doi: 10.1016/j.engappai.2022.105139.

[34] F. Uddin, N. Riaz, A. Manan, I. Mahmood, O.-Y. Song, A. J. Malik, and A. A. Abbasi, "An improvement to the 2-opt heuristic algorithm for approximation of optimal TSP tour," *Appl. Sci.*, vol. 13, no. 12, p. 7339, 2023, doi: 10.3390/app13127339.

[35] X. Li, J.-Y. Li, C. Chen, Z.-H. Zhan, S. Kwong, and J. Zhang, "Matrix-based ant colony system for traveling salesman problem," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, 2024, pp. 1358–1363, doi: 10.1109/SMC54092.2024.10831943.

[36] T. Huang, Z.-Q. Zhang, Y.-J. Gong, and J. Liu, "nLKH-ACS: A niching Lin-Kernighan–Helsgaun-based ant colony system for multisolution traveling salesman problems," *IEEE Trans. Evol. Comput.*, vol. 29, no. 6, pp. 2596–2610, 2024, doi: 10.1109/TEVC.2024.3507777.

[37] M. Lin and L. P. Schaposnik, “A carbon aware ant colony system for the sustainable generalized traveling salesman problem,” *Sci. Rep.*, vol. 15, p. 21928, 2025, doi: 10.1038/s41598-025-08276-9.

[38] B. Sun, W.-J. Yu, Z.-J. Wang, and J.-Y. Li, “Ant colony optimization for multiple traveling salesmen problem with revisitable cities,” in *Proc. IEEE Int. Conf. Mach. Intell. Theory Appl. (MiTA)*, 2024, pp. 1–6, doi: 10.1109/MiTA60795.2024.10751693.

[39] B. Behdani and J. C. Smith, “An integer-programming-based approach to the close-enough traveling salesman problem,” *INFORMS J. Comput.*, vol. 26, no. 3, pp. 415–432, 2014, doi: 10.1287/ijoc.2013.0574.

[40] B. Yao, C. Chen, X. Song, and X. Yang, “Fresh seafood delivery routing problem using an improved ant colony optimization,” *Ann. Oper. Res.*, vol. 273, no. 1–2, pp. 163–186, 2019, doi: 10.1007/s10479-017-2531-2.